INSTITUTO MAUÁ DE TECNOLOGIA

ENGENHARIA DA COMPUTAÇÃO

FERNANDO GODOI GRINEVICIUS; RA: 22-00832-2

GABRIEL BARROCHELO; RA: 22.10193-4

JONAS FERNANDO DA SILVA EBOLI MACHADO; RA: 22.00910-8

MATHEUS ANTÔNIO DA LUZ CARDOSO; RA: 22.01059-9

RUTH RAMOS ROMEU; RA: 22.01003-3

VINÍCIUS ELOY ARAUJO; RA: 22.01026-2

Aktooh

Projeto Semestral de linguagens de programação – 2° Semestre

SÃO CAETANO DO SUL

2025

FERNANDO GODOI GRINEVICIUS; RA: 22-00832-2

GABRIEL BARROCHELO; RA: 22.10193-4

JONAS FERNANDO DA SILVA EBOLI MACHADO; RA: 22.00910-8

MATHEUS ANTÔNIO DA LUZ CARDOSO; RA: 22.01059-9

RUTH RAMOS ROMEU; RA: 22.01003-3

VINÍCIUS ELOY ARAUJO; RA: 22.01026-2

Aktooh

Projeto Semestral de linguagens de programação – 2° Semestre

Projeto Semestral para a disciplina de linguagens de programação apresentado a Instituto Mauá de Tecnologia – IMT como requisito parcial para conclusão da disciplina.

Professor Robson Calvetti.

SÃO CAETANO DO SUL

2025

RESUMO

Este relatório apresenta o desenvolvimento do Projeto II da disciplina de Linguagem de Programação I, cujo objetivo foi evoluir um sistema de quiz interativo produzido no primeiro semestre. Nesta nova fase, o sistema foi ampliado para incluir uma arquitetura cliente-servidor, suporte a múltiplos idiomas, manipulação de arquivos texto, criptografia, componentes sofisticados de interface gráfica e integração completa com banco de dados MySQL.

O servidor, implementado em Java, é responsável por receber solicitações do cliente, processar informações relacionadas ao quiz e interagir com o banco de dados, utilizando JDBC. Já o cliente, também em Java, apresenta ao usuário uma interface gráfica construída com Swing, permitindo execução do quiz, edição de perguntas e visualização de resultados. Comunicação entre cliente e servidor é realizada por meio de sockets TCP com suporte a threads, possibilitando múltiplas conexões simultâneas.

Os resultados demonstram que o sistema atende aos requisitos propostos e apresenta arquitetura modular, segura e extensível. O trabalho reforça conceitos fundamentais de programação, engenharia de software e redes.

Palavras-chave: Java, Sockets, Cliente-Servidor, Swing, MySQL, Criptografia, Internacionalização.

ABSTRACT

This report presents the development of Project II for the Programming Language I course, whose main objective was to upgrade a quiz system originally created during the first semester. In this second phase, the system was expanded to include a client-server architecture, multilingual support, text-file manipulation, cryptography, advanced Swing graphical components, and complete MySQL database integration.

The server, implemented in Java, receives client requests, processes quiz-related data and interacts with the database through JDBC. The client application, also written in Java, provides a graphical interface with features such as quiz execution, question editing, and results visualization. Communication between client and server is performed through TCP sockets with multi-thread support, enabling simultaneous connections.

Results indicate that the system fulfills all proposed requirements and presents a modular, secure and extensible architecture. This work reinforces essential concepts in programming, software engineering and computer networking.

Keywords: Java, Sockets, Client-Server, Swing, MySQL, Cryptography, Internationalization.

SUMÁRIO

RESUMO 3
ABSTRACT 4
1. INTRODUÇÃO 6
2. FUNDAMENTAÇÃO TEÓRICA 7
2.1 Programação Orientada a Objetos 7
2.2 Comunicação Cliente-Servidor 7
2.3 JDBC e Persistência de Dados 7
2.4 Interfaces Gráficas com Swing 7
2.5 Internacionalização 8
2.6 Criptografia 8
2.7 Manipulação de Arquivos 8
3. METODOLOGIA 9
4. DESENVOLVIMENTO DO SISTEMA 11
4.1 Arquitetura do Sistema 11
4.2 Implementação Cliente-Servidor 11
4.3 Módulo de Chat 12
4.4 Integração com o Banco de Dados 13
4.5 Interface Gráfica 14
4.6 Internacionalização 15
4.7 Criptografia 16
4.8 Manipulação de Arquivos 16
5. RESULTADOS E DICUSÃO 18
5.1 Funcionamento Geral do Sistema 18
5.2 Testes das Funcionalidades do Quiz 18
5.3 Integração Cliente-Servidor 19
5.4 Avaliação do Módulo de Chat 19
5.5 Exportação e Segurança dos Resultados 20
6. CONCLUSÃO 21
7. REFERÊNCIAS 22

1. INTRODUÇÃO

Este relatório tem como objetivo apresentar o desenvolvimento do Projeto II da disciplina ECM251 – Linguagem de Programação I. O trabalho consiste na expansão de um sistema de quiz desenvolvido no primeiro semestre, agora adaptado para uma arquitetura cliente-servidor, além da implementação de novas funcionalidades como internacionalização, manipulação de arquivos texto, criptografia, uso de componentes avançados da biblioteca Swing e integração com banco de dados MySQL.

Os principais objetivos foram implementar um servidor capaz de lidar com múltiplas requisições simultâneas utilizando threads, estabelecer comunicação por sockets, aprimorar a interface e garantir persistência segura dos dados. O relatório descreve o embasamento teórico, metodologia adotada, desenvolvimento, testes e resultados obtidos.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Programação Orientada a Objetos

A Programação Orientada a Objetos (POO) foi amplamente utilizada no desenvolvimento do sistema, estruturando o código em classes que representam conceitos reais do domínio da aplicação. No projeto, a classe User encapsula informações sobre os usuários do sistema, incluindo identificação, nome, senha e tipo (aluno ou professor). Já a classe Question representa cada pergunta cadastrada no banco de dados, contendo o enunciado, alternativas e a resposta correta, enquanto a classe QuestionForm auxilia na construção de formulários de perguntas que são exibidos durante a execução do quiz.

2.2 Comunicação Cliente-Servidor

A arquitetura cliente-servidor consiste na separação entre:

- **Servidor:** responsável pela lógica de negócio e persistência
- **Cliente:** responsável pela interface e interação com o usuário

A comunicação foi implementada utilizando **sockets TCP**, permitindo troca de mensagens estruturadas entre aplicações.

2.3 JDBC e Persistência de Dados

O Java Database Connectivity (JDBC) é utilizado para estabelecer conexões com o MySQL, enviar comandos SQL e tratar resultados. A classe ConnFactory abstrai a criação de conexões, enquanto CrudBD encapsula operações CRUD.

2.4 Interfaces Gráficas com Swing

Swing fornece componentes gráficos robustos como:

- JTable
- JComboBox
- JMenu
- JDialog
- JButton

- JFrame

O sistema utiliza esses recursos para criar telas de navegação, edição e visualização.

2.5 Internacionalização

A internacionalização (i18n) foi implementada com a classe ResourceBundle, permitindo troca de idioma sem reiniciar a aplicação.

2.6 Criptografia

Para proteção dos resultados consolidados dos alunos acessados pelo professor, o projeto inclui o uso de criptografia simétrica. As classes do pacote Arquivos, em especial AESKeyStore e CryptoAESGCM, geram e armazenam uma chave AES e realizam a criptografia e descriptografia de arquivos utilizando o modo GCM (Galois/Counter Mode), que fornece confidencialidade e integridade dos dados.

2.7 Manipulação de Arquivos

A manipulação de arquivos desempenha um papel fundamental em sistemas que necessitam armazenar informações de forma externa e independente do banco de dados. Em geral, esse recurso permite exportar registros, gerar relatórios e manter dados acessíveis mesmo fora da aplicação. Arquivos texto, em especial, são amplamente utilizados pela sua simplicidade, portabilidade e fácil leitura por usuários ou outros sistemas.

Além disso, projetos que lidam com dados sensíveis podem adotar mecanismos de proteção baseados em arquivos adicionais, como aqueles destinados ao armazenamento de chaves criptográficas. Esses arquivos permitem que a aplicação mantenha informações de segurança separadas do código-fonte, reduzindo riscos de exposição e fortalecendo a integridade do sistema. Assim, a manipulação de arquivos se torna um componente essencial tanto para funcionalidades de exportação quanto para suporte a técnicas de segurança, servindo de base para recursos mais avançados descritos nas seções posteriores.

3. METODOLOGIA

O desenvolvimento do projeto foi conduzido de forma incremental e colaborativa, seguindo uma abordagem prática alinhada ao conteúdo apresentado ao longo do semestre. Inicialmente, foi realizada uma revisão completa do sistema desenvolvido no Projeto I, com o objetivo de identificar partes do código que poderiam ser reaproveitadas, bem como os pontos que exigiriam reestruturação para a nova etapa, especialmente considerando os requisitos de cliente-servidor, internacionalização, criptografia, arquivos texto e componentes gráficos avançados. Após essa análise, o grupo definiu a divisão de tarefas, atribuindo a cada integrante responsabilidades específicas relacionadas à interface gráfica, lógica de negócio, integração com banco de dados e preparação para a comunicação via sockets.

A implementação foi realizada utilizando a IDE Visual Studio Code, configurada com as extensões necessárias para suporte ao Java. O controle de versão ocorreu por meio da plataforma GitHub, o que permitiu que todos os membros do grupo trabalhassem simultaneamente no código, criassem branches, resolvessem conflitos e registrassem as evoluções do sistema de forma organizada. Paralelamente, foram utilizados os arquivos de referência fornecidos pelo professor, além do material teórico da disciplina, para orientar decisões sobre estrutura do código, boas práticas de programação e padronização das classes.

Para atender aos requisitos funcionais impostos pelo Projeto II, iniciamos pela criação e reorganização das interfaces gráficas em Swing, como as classes GuiUser, GuiCentralAluno, GuiCentralProfessor, GuiSetQuiz, GuiEditQuestions e GuiViewResults, garantindo que cada tela fosse construída manualmente, sem uso de ferramentas automáticas. Em seguida, implementou-se a camada de persistência, baseada nas classes ConnFactory e CrudBD, responsáveis por estabelecer a conexão via JDBC com o banco de dados MySQL e realizar as operações CRUD necessárias para manipulação de usuários, perguntas, quizzes e resultados.

Para concluir o desenvolvimento, foi realizada a implementação completa da comunicação cliente-servidor utilizando sockets TCP, conforme os exemplos estudados em aula. A estrutura foi organizada de modo que o cliente estabelecesse conexão com o servidor por meio de uma classe dedicada, enviando comandos de

texto padronizados, enquanto o servidor operava em uma thread principal responsável por aceitar novas conexões e delegar cada cliente a uma thread individual. Esse servidor, por sua vez, fazia a mediação entre o cliente e o banco de dados, encaminhando as solicitações recebidas para os métodos da classe CrudBD e retornando ao cliente apenas os dados processados. Após essa etapa, foram realizados testes integrados para validar o fluxo completo de comunicação, garantindo que operações como login, carregamento de perguntas, envio de respostas e consulta de resultados funcionassem corretamente entre cliente, servidor e banco de dados. Concluídas todas as implementações, iniciou-se a elaboração do relatório técnico, seguindo as normas da ABNT e documentando todas as fases do projeto.

4. DESENVOLVIMENTO DO SISTEMA

4.1 Arquitetura do Sistema

A arquitetura do sistema foi organizada em camadas, seguindo um modelo modular que separa interface, lógica de negócio, comunicação e persistência. Essa divisão permite que cada parte do sistema evolua de forma independente e garante maior clareza na manutenção do código.

A camada de **apresentação** é responsável por exibir informações aos usuários e capturar suas ações. Ela interage diretamente com a camada de lógica, que centraliza as regras do sistema, valida operações e organiza o fluxo de dados entre os componentes.

A camada de **comunicação** implementa o modelo Cliente-Servidor por meio de sockets, permitindo que as solicitações do cliente sejam enviadas ao servidor, que processa comandos, acessa o banco de dados e retorna as respostas adequadas.

Por fim, a camada de **persistência** é responsável pelo armazenamento definitivo das informações, utilizando um banco de dados relacional. A lógica de acesso a dados permanece encapsulada em classes específicas, garantindo que outras partes do sistema utilizem os dados sem conhecimento dos detalhes de implementação.

Essa organização em camadas contribui para um sistema mais estruturado, reutilizável e preparado para futuras expansões.

4.2 Implementação Cliente-Servidor

A implementação do modelo Cliente-Servidor foi projetada para centralizar o processamento do sistema e organizar o fluxo de informações entre os usuários e o banco de dados. Nesse modelo, o **cliente** representa a aplicação utilizada pelo aluno ou professor, responsável apenas pela interação com o usuário e pela emissão de solicitações. Já o **servidor** atua como o núcleo do sistema, recebendo comandos, processando-os e retornando os resultados de forma padronizada.

A comunicação ocorre por meio de **sockets TCP**, seguindo um protocolo textual simples baseado em comandos e parâmetros. Cada comando enviado pelo

cliente é interpretado pelo servidor, que executa a operação correspondente na camada de persistência e devolve uma resposta adequada. Para permitir múltiplos acessos simultâneos, o servidor utiliza threads, garantindo que cada cliente conectado seja atendido de forma independente.

Essa abordagem garante um fluxo de dados organizado e seguro, separando responsabilidades e permitindo que o cliente permaneça leve, enquanto o servidor concentra toda a lógica de validação, gerenciamento de quizzes, manipulação de perguntas e acesso ao banco de dados. Além disso, a adoção do modelo Cliente-Servidor torna o sistema escalável e facilita futuras expansões, como uso em rede local, hospedagem remota ou integração com outras aplicações.

4.3 Módulo de Chat

O sistema também inclui um módulo de chat integrado, desenvolvido com base no modelo Cliente-Servidor utilizando sockets TCP. Esse recurso permite que usuários conectados troquem mensagens em tempo real, ampliando a interatividade durante a utilização do quiz. Para isso, foram implementadas três classes principais: **ChatServer**, **ClientHandler** e **ChatClient**, cada uma com responsabilidades específicas dentro do fluxo de comunicação.

A classe **ChatServer** atua como servidor central do módulo de chat. Ela escuta continuamente em uma porta TCP dedicada (porta 33444 no projeto) e aceita novas conexões à medida que os usuários se conectam. Para cada cliente conectado, o servidor cria uma instância de *ClientHandler*, mantendo também uma lista sincronizada de todos os usuários ativos. Essa lista é utilizada para gerenciar o envio de mensagens por broadcast, garantindo que qualquer mensagem recebida seja distribuída imediatamente a todos os participantes do chat.

A classe **ClientHandler** é responsável por tratar individualmente cada conexão. Ela recebe e interpreta mensagens enviadas por um cliente específico e as encaminha adequadamente para o servidor, que então distribui o conteúdo aos demais usuários conectados. Além disso, o *ClientHandler* participa da atualização da lista de usuários online, permitindo que a interface do chat mostre em tempo real quem está conectado.

Já o **ChatClient** representa o cliente do chat integrado à interface gráfica do sistema. Ele estabelece conexão com o servidor via socket TCP e disponibiliza ao usuário um painel dedicado com área de exibição de mensagens, campo de entrada para envio de texto e indicadores de usuários online. O cliente também recebe mensagens especiais no formato USERS:, enviadas pelo servidor, que permitem atualizar automaticamente a lista de participantes ativos.

Esse módulo adiciona uma camada adicional de interação ao sistema, permitindo comunicação entre os usuários durante a execução das atividades e proporcionando uma experiência mais dinâmica e colaborativa dentro da aplicação.

4.4 Integração com o Banco de Dados

O sistema utiliza um banco de dados MySQL acessado via JDBC. As operações são encapsuladas na classe **CrudBD.java**, que contém métodos específicos para manipulação das tabelas do sistema. A obtenção das conexões é feita através da classe **ConnFactory.java**, que centraliza a configuração de URL, usuário e senha do banco.

4.4.1 Estrutura do Banco

Foram utilizadas as seguintes tabelas:

- **users** – Armazena informações de usuários (id, nome, senha, tipo, pontuação).
- **questions** – Armazena perguntas e alternativas.
- **quizzes** – Cadastro de quizzes disponíveis.
- **quiz_questions** – Relação entre quizzes e perguntas.
- **results** – Armazena resultados de alunos.

4.4.2 Operações CRUD

A classe CrudBD possui métodos implementados com PreparedStatement, tais como:

- addQuestion(Question q)
- updateQuestion(Question q)
- deleteQuestion(int id)

- getQuestions()
- addUser(User u)
- validateLogin(String nome, String senha)
- saveResult(int userId, int quizId, int score)

Esses métodos são acessados indiretamente pelas telas via Cliente-Servidor, garantindo consistência no fluxo de dados.

4.5 Interface Gráfica

A interface gráfica do sistema foi desenvolvida integralmente em Java Swing, sem uso de ferramentas automáticas, e organizada em várias telas independentes, cada uma responsável por uma parte específica do funcionamento do aplicativo.

- **GuiUser.java**: Tela inicial do sistema. Responsável pelo login e cadastro de usuários, além de direcionar o usuário ao painel de professor ou aluno conforme o tipo de conta.
- **GuiSelectIdioma.java**: Painel onde o usuário seleciona o idioma do quiz.
- **GuiCentralAluno.java**: Painel principal do aluno. Permite selecionar um quiz, iniciar sua execução, visualizar resultados anteriores e exportar dados.
- **GuiCentralProfessor.java**: Painel principal do professor. Fornece acesso à edição de perguntas, visualização de resultados dos alunos e ferramentas de gerenciamento dos quizzes.
- **GuiEditQuestions.java**: Tela onde o professor pode criar, editar e excluir perguntas. Envia operações de CRUD para o servidor/banco de dados conforme a ação desejada.
- **GuiSetQuiz.java**: Tela onde o aluno responde ao quiz. Apresenta as perguntas e alternativas, coleta as respostas e envia o resultado para registro.
- **GuiViewResults.java**: Tela destinada aos alunos para visualização de suas próprias notas em quizzes anteriores. Mostra os dados em uma tabela organizada (JTable).

- **GuiViewResultsProfessor.java**: Versão da tela de resultados voltada para o professor. Exibe o desempenho dos alunos, podendo filtrar por quiz ou outras informações relevantes.
- **GuiSelectIdioma.java**: Tela simples que permite ao usuário escolher o idioma do sistema. Atualiza automaticamente todos os textos das interfaces por meio de arquivos ResourceBundle.
- **GuiQuestions.java**: Componente auxiliar utilizado para estruturar visualmente perguntas e alternativas, sendo usado em telas como a de edição ou execução de quiz.
- **QuestionForm.java**: Componente que representa uma única pergunta dentro do quiz, contendo enunciado, alternativas e método para verificar se a resposta marcada está correta.

4.6 Internacionalização

A internacionalização do sistema foi implementada utilizando o mecanismo de **ResourceBundle** do Java, permitindo que todos os textos da interface gráfica sejam carregados de arquivos externos de acordo com o idioma selecionado pelo usuário. Para isso, foram criados arquivos .properties específicos para cada idioma suportado, contendo pares de chave–valor que representam todas as mensagens e rótulos utilizados na aplicação.

Cada tela da aplicação consulta essas traduções por meio de chamadas como rb.getString("label.login"), garantindo que a interface possa ser exibida integralmente no idioma escolhido sem modificações no código-fonte. A tela **GuiSelectIdioma.java** é responsável pela seleção dinâmica do idioma por meio de um JComboBox. Ao mudar a opção, o sistema recarrega automaticamente o ResourceBundle correspondente e atualiza os textos exibidos.

Essa abordagem modular torna simples a manutenção das traduções e permite adicionar novos idiomas no futuro apenas criando mais arquivos .properties, sem a necessidade de alterar a lógica das telas. Além disso, a internacionalização amplia a acessibilidade do sistema, tornando-o adequado para diferentes públicos e alinhado às boas práticas de desenvolvimento de software.

4.7 Criptografia

A criptografia foi incorporada ao sistema com o objetivo de aumentar a segurança na manipulação de dados sensíveis, especialmente informações relacionadas a usuários e arquivos gerados. A implementação utiliza o algoritmo **AES (Advanced Encryption Standard)** em modo GCM, que é reconhecido por oferecer confidencialidade e autenticação dos dados em uma única operação. Para isso, foram desenvolvidas classes responsáveis tanto pelo processo de cifragem quanto pela gestão da chave criptográfica.

Os arquivos .key utilizados no sistema armazenam a chave secreta necessária para criptografar e descriptografar conteúdos, evitando que a chave fique exposta diretamente no código-fonte. Durante a execução, os métodos de criptografia acessam esse arquivo para garantir que o mesmo padrão de cifragem seja aplicado em diferentes execuções. Esse mecanismo é utilizado principalmente na geração de arquivos texto protegidos e em operações internas que exigem maior segurança, permitindo que o sistema siga boas práticas de proteção de dados sem comprometer desempenho ou usabilidade.

4.8 Manipulação de Arquivos

O sistema também implementa funcionalidades para criação e manipulação de arquivos texto, oferecendo ao usuário a possibilidade de exportar informações relevantes, como resultados de quizzes, registros de desempenho e relatórios simples. Para isso, foram desenvolvidas classes específicas que geram arquivos .txt a partir dos dados presentes no sistema, utilizando estruturas como FileWriter e BufferedWriter para garantir uma escrita eficiente e organizada.

Esses arquivos podem ser utilizados por alunos e professores para armazenar localmente seus resultados, o que adiciona praticidade e permite consulta externa ao sistema. Além disso, os textos gerados podem ser protegidos pelos mecanismos de criptografia descritos anteriormente, garantindo segurança adicional quando necessário. A manipulação de arquivos texto também torna o sistema mais completo e funcional, contribuindo para atender aos requisitos da disciplina e ampliar o nível de profissionalismo da aplicação.

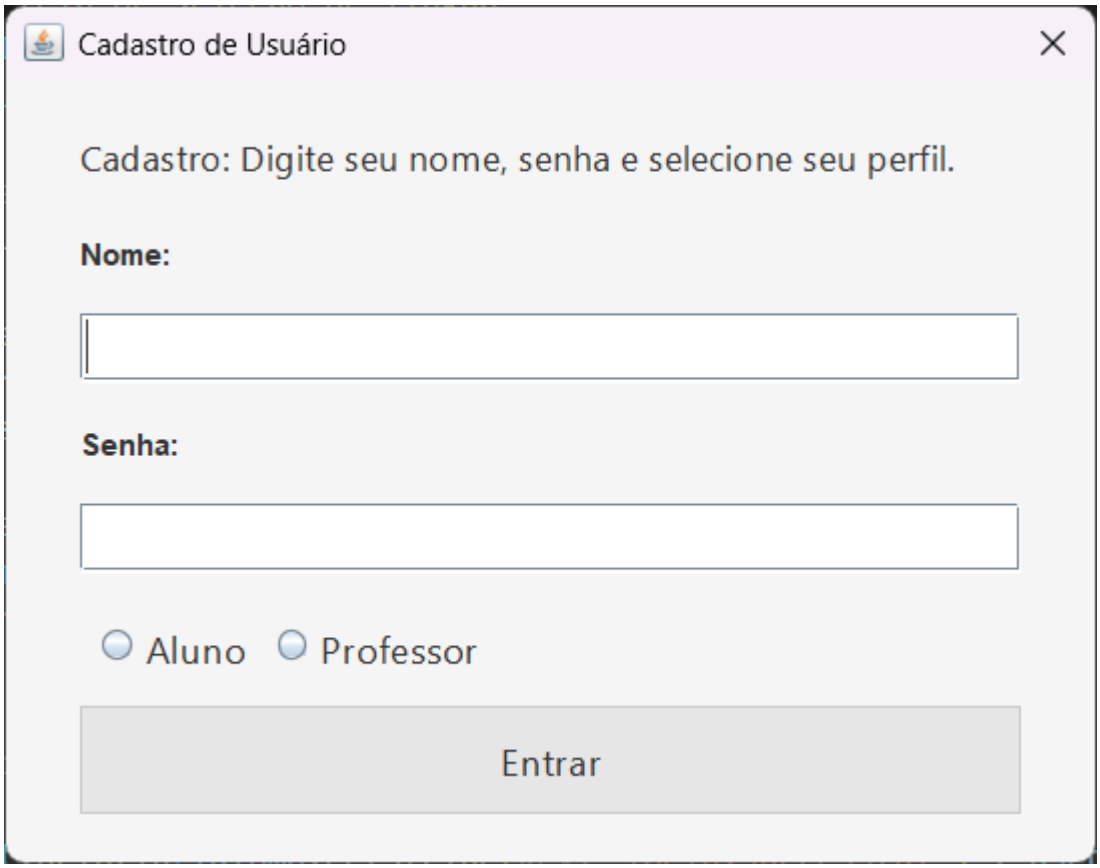
- **AESKeyStore:** Gera e carrega uma chave AES usada na criptografia dos arquivos de resultados.
- **CryptoAESGCM:** Realiza a criptografia (encryptFile) e descriptografia (decryptFile) de arquivos utilizando AES/GCM/NoPadding.
- **CriaTxtE:** Gera o arquivo resultados.txt, contendo dados de quiz e pontuação voltados ao aluno (por exemplo, ranking ou resumo de desempenho).
- **CriaTxtP:** Gera o arquivo resultadosProfessor.txt com informações detalhadas (ID do aluno, nome do quiz, pontuação), e em seguida criptografa esse arquivo, gerando resultadosProfessor.txt.enc, ampliando a segurança dessas informações.
- **Descriptografa:** Classe de apoio para descriptografar o arquivo protegido, permitindo ao professor recuperar os resultados quando necessário.

5. RESULTADOS E DICUSÃO

5.1 Funcionamento Geral do Sistema

O sistema final demonstrou operação estável e consistente, contemplando as principais funcionalidades previstas: autenticação de usuários, execução de quizzes, gerenciamento de perguntas, exportação de resultados, chat integrado e comunicação Cliente-Servidor. Todos os módulos funcionam de forma integrada, permitindo interação fluida entre interface gráfica, servidor central e banco de dados.

Figura 1 – Tela Inicial de Login (GuiUser)



Fonte: Autores

5.2 Testes das Funcionalidades do Quiz

Durante os testes, foi possível validar o fluxo completo de execução de um quiz, desde a seleção do questionário até o registro do resultado no banco de dados.

As perguntas foram corretamente carregadas, exibidas ao aluno e avaliadas pela lógica do sistema. Os professores puderam criar, atualizar e excluir questões sem inconsistências.

Figura 2 – Exemplo de pergunta do quiz

Fonte: Autores

5.3 Integração Cliente-Servidor

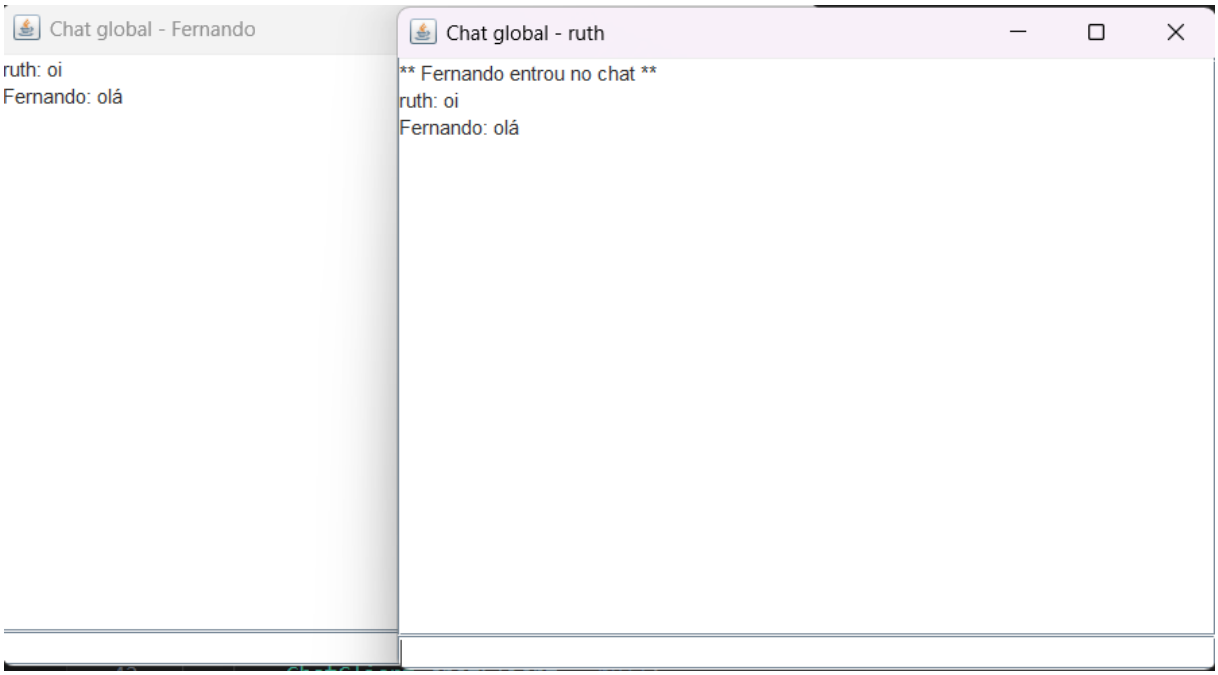
Os testes demonstraram que o servidor foi capaz de atender múltiplos clientes simultaneamente, mantendo estabilidade e tempo de resposta adequado. Os comandos enviados pelo cliente foram processados corretamente pelo servidor, que encaminhou as respostas de acordo com o protocolo estabelecido. Não foram identificados travamentos ou perda de conexão durante o uso normal.

5.4 Avaliação do Módulo de Chat

O módulo de chat apresentou funcionamento satisfatório, permitindo comunicação em tempo real entre usuários conectados ao servidor. As mensagens foram corretamente

distribuídas por broadcast; a lista de usuários online foi atualizada conforme conexões e desconexões ocorreram. O chat mostrou baixa latência e boa responsividade mesmo com vários clientes ativos.

Figura 3 – Tela do chat dentro do Quiz App



Fonte: Autores

5.5 Exportação e Segurança dos Resultados

A funcionalidade de exportação para arquivos texto apresentou boa estabilidade, gerando documentos organizados contendo pontuação, datas e informações relevantes. Quando combinada ao módulo de criptografia (AES/GCM), mostrou-se robusta e segura, permitindo que o professor armazenasse resultados protegidos contra acesso indevido.

6. CONCLUSÃO

O projeto de Sistema de Quiz Multilíngue com Chat Integrado alcança o objetivo de reunir, em uma única aplicação, diversos conceitos estudados na disciplina de Linguagens de Programação, como:

- Aplicação prática de orientação a objetos em Java.
- Uso de interfaces gráficas com Swing.
- Integração com banco de dados relacional via JDBC.
- Implementação de comunicação em rede com sockets TCP.
- Noções de segurança de dados mediante o uso de criptografia simétrica.
- Internacionalização da interface e das perguntas por meio de ResourceBundle e tabelas específicas por idioma.

Algumas possíveis melhorias e extensões para o projeto incluem:

- Criação de um sistema de cadastro e autenticação de professores com perfis mais detalhados.
- Interface para o professor cadastrar e editar perguntas diretamente pela aplicação, sem necessidade de alterar o banco via SQL.
- Persistência de logs de chat e possibilidade de chats privados ou por sala.
- Implementação de relatórios gráficos de desempenho (por exemplo, evolução da pontuação ao longo do tempo).
- Migração da interface para tecnologias web ou JavaFX, visando maior modernização visual.
- Implementação de mecanismos mais avançados de segurança, como hashing de senhas no banco de dados.

7. REFERÊNCIAS

BRASIL. **Associação Brasileira de Normas Técnicas.** NBR 6023: Informação e documentação – Referências – Elaboração. Rio de Janeiro, 2018.

DEITEL, Paul; DEITEL, Harvey. **Java: Como Programar.** 10. ed. São Paulo: Pearson, 2016.

ECKEL, Bruce. **Thinking in Java.** 4. ed. Upper Saddle River: Prentice Hall, 2006.

HORSTMANN, Cay S.; CORNELL, Gary. **Core Java – Volume I: Fundamentals.** 11. ed. Pearson, 2018.

HORSTMANN, Cay S.; CORNELL, Gary. **Core Java – Volume II: Advanced Features.** 11. ed. Pearson, 2018.

ORACLE. **Java Platform, Standard Edition – Documentation.** Disponível em: https://docs.oracle.com/javase/. Acesso em: 24 nov. 2025.

ORACLE. **Java Swing Tutorial.** Disponível em: https://docs.oracle.com/javase/tutorial/uiswing/. Acesso em: 24 nov. 2025.

ORACLE. **Java Database Connectivity (JDBC) Guide.** Disponível em: https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/. Acesso em: 24 nov. 2025.

STALLINGS, William. **Cryptography and Network Security: Principles and Practice.** 7. ed. Pearson, 2017.

TANENBAUM, Andrew S.; WETHERALL, David. **Redes de Computadores.** 5. ed. Rio de Janeiro: Pearson, 2011.

MYSQL. **MySQL Documentation – Developer Guide.** Disponível em: https://dev.mysql.com/doc/. Acesso em: 24 nov. 2025.

KURZ, Oliver. **Java Sockets: Network Programming with Java.** Disponível em: https://docs.oracle.com/javase/tutorial/networking/. Acesso em: 24 nov. 2025.

CALVETTI, Professor. **Material da disciplina – Cliente/Servidor com Java Sockets.** Instituto Mauá de Tecnologia, 2025. Apostila fornecida em sala de aula.